# HANZO BASE

## The Single-Binary Runtime for Sovereign Applications

Zach Kelling

*Hanzo AI*

March 2026

**Abstract**

We present Hanzo Base, the application runtime for sovereign software. Base is not a backend framework. It is the personal computer of Web5—a single Go binary that packages an encrypted embedded database (SQLite with sqlcipher), a realtime API layer, CRDT-based collaboration, a JavaScript plugin runtime (Goja VM), Hanzo IAM authentication, and field-level KMS encryption into one process. Where Firebase assumes a cloud provider, Supabase assumes PostgreSQL, and PocketBase assumes a single user, Base assumes nothing. It runs on a laptop, a phone, a Raspberry Pi, or a Kubernetes pod with the same binary and the same API surface. Data stays local by default. Replication is opt-in. Chain anchoring is available but never required. We describe the architecture, the data plane (local vault, provider-optional replication, Lux receipt anchoring), Base Functions (TypeScript in Goja, event-driven, zero cold start), Base Collections (schemaless records with access rules and field-level encryption), Base Sync (CRDT log, conflict-free merge, offline-first, peer-to-peer or server-relay), and integration with Lux Network chains (K-Chain key policy, I-Chain identity, T-Chain threshold controls). Benchmarks on an M4 MacBook show 10,400 writes/sec for encrypted collections and sub-millisecond read latency. In production since December 2022, Base serves as the default runtime for 2,800+ applications across the Hanzo ecosystem.

# 1 Introduction

Software is not sovereign. The applications people depend on daily—messaging, notes, banking, health records—exist as tenants on infrastructure controlled by third parties. The user's data lives in a provider's database, encrypted with the provider's keys, governed by the provider's terms. If the provider disappears, so does the application. If the provider changes policy, the user complies or loses access. This is not ownership. It is tenancy.

The problem is architectural. Cloud-native design treats the network as a given: every read is a round-trip, every write is a permission request, every query is metered. The application is a thin client to someone else's computer. The alternative—self-hosting—requires expertise that excludes all but a small technical minority. There is no middle ground.

Base is the middle ground.

**Definition 1** (Sovereign Application)**.** *An application is sovereign if: (i) the user controls the encryption keys for all stored data, (ii) the application functions without any network connection, (iii) replication to any remote host is opt-in and reversible, and (iv) no third party can unilaterally revoke access to the application or its data.*

Base achieves sovereignty by inverting the cloud model. Instead of a thin client talking to a remote server, Base is the server—running locally, owning its own database, generating its own keys. Remote services (cloud storage, chain anchoring, peer sync) are optional extensions, never requirements.

## 1.1 Contributions

1. A single-binary application runtime with embedded encrypted database, realtime APIs, authentication, and plugin system (§2).

2. A data plane architecture where local storage is primary and remote replication is provider-optional (§3).

3. Base Functions: TypeScript execution in an embedded Goja VM with zero cold start and event-driven hooks (§4).

4. Base Collections: schemaless records with row-level access rules and field-level KMS encryption (§5).

5. Base Sync: CRDT-based conflict-free replication supporting offline-first, peer-to-peer, and server-relay topologies (§6).

6. Integration with Lux Network for key policy, identity, and threshold controls (§7).

7. Performance evaluation showing 10,400 encrypted writes/sec and sub-millisecond reads on commodity hardware (§10).

## 2 Architecture

Base is a single statically-linked Go binary. It embeds all components: the database engine, the HTTP server, the WebSocket server, the JavaScript VM, the authentication system, the admin dashboard, and the migration runner. There is no external dependency at runtime—no PostgreSQL, no Redis, no message broker, no reverse proxy.

### 2.1 System Components

| Component | Implementation | Function |
|---|---|---|
| Database | SQLite + sqlcipher | Encrypted embedded storage |
| HTTP API | Go net/http | REST endpoints, admin UI |
| Realtime | WebSocket | Live subscriptions, change feeds |
| Plugin VM | Goja (ES2023) | TypeScript/JS function runtime |
| Auth | Hanzo IAM (OIDC) | Federated identity, org scoping |
| Admin | Embedded SPA | Collection manager, log viewer |
| Migration | SQL + Go | Schema versioning, auto-migration |
| KMS Client | hanzoai/kms | Field-level encryption keys |

Table 1: Base runtime components. All are compiled into a single binary.

### 2.2 Binary Size and Startup

The Base binary is 28 MB (stripped, compressed with UPX: 12 MB). Startup time from cold to serving HTTP requests is 45 ms on an M4 MacBook and 120 ms on a Raspberry Pi 4. This is not a framework that boots a JVM, connects to three external services, and runs database migrations. It is a process that opens a file and starts listening.

### 2.3 Deployment Modes

Base supports three deployment modes with identical API surfaces:

1. **Local**: Single binary on the user's machine. Data in `~/.base/data.db`. No network required.

2. **Server**: Single binary on a VPS or bare-metal host. Exposed via DNS. Optional TLS termination via Hanzo Ingress.

3. **Cluster**: Multiple Base instances behind a load balancer, sharing a PostgreSQL database for write coordination. SQLite remains the local cache on each node.

The critical property: applications written for local mode work in cluster mode without modification. The API is the same. The data model is the same. The auth flow is the same. Only the storage backend changes, and that change is a single environment variable.

# 3 Base as Data Plane

Base treats local storage as the primary data plane. This is the opposite of every cloud-native system, where the network database is primary and the client is a cache. In Base, the local SQLite database is the source of truth. Remote storage is a replica.

## 3.1 Local Vault

Every Base instance maintains a local encrypted database using SQLite compiled with the sqlcipher extension. The encryption key is derived from the user's master key via Argon2id:

$$k_{\text{db}} = \text{Argon2id}(k_{\text{master}}, \text{salt}, t = 3, m = 64\text{MB}, p = 4) \tag{1}$$

The database file is AES-256-CBC encrypted at the page level. An attacker who obtains the file without the key sees random bytes. The key never leaves the device unless the user explicitly exports it (for backup or device migration).

## 3.2 Provider-Optional Replication

Replication in Base is a directed graph of sync targets. Each target is a remote Base instance, an S3-compatible bucket, or a Lux chain endpoint. The user configures zero or more targets:

- **Zero targets**: Fully offline. Data exists only on the local device. Backup is the user's responsibility.
- **One target**: Simple backup. Changes replicate to a single remote. Recovery is possible from the remote.
- **Multiple targets**: Redundant replication. Changes fan out to multiple remotes. Availability survives any single target failure.

No target is privileged. There is no "primary" remote. The local database is always authoritative. If a remote diverges, the CRDT merge (§6) resolves the conflict deterministically.

## 3.3 Chain Anchoring

Base optionally anchors data integrity proofs to the Lux Network. Anchoring does not store data on-chain—it stores a Merkle root of the local database state, creating an immutable timestamp proof.

$$\text{anchor}(t) = \text{SHA3-256}\big(\text{root}(\text{MerkleTree}(D_t))\big) \tag{2}$$

where $D_t$ is the set of all records at time $t$. The anchor transaction is submitted to the Lux T-Chain (transaction chain), providing sub-second finality and costing less than 0.001 LUX per anchor. Applications that require tamper-evidence (medical records, legal documents, audit logs) enable anchoring. Applications that do not need it pay nothing.

# 4 Base Functions

Base Functions are TypeScript functions that run inside the Base process. They are not Lambda functions. They are not containers. They are JavaScript closures executed in an embedded Goja VM, sharing the same process and the same database connection as the rest of Base.

## 4.1 Execution Model

The Goja VM implements ECMAScript 2023. TypeScript is transpiled to JavaScript at deploy time (via esbuild, also embedded in the binary). Functions execute in response to events:

- **Before hooks**: Run before a database operation (create, update, delete). Can modify the record or reject the operation.
- **After hooks**: Run after a database operation. Used for side effects (notifications, audit logs, cache invalidation).
- **Routes**: Custom HTTP endpoints registered in the Base router. Full access to request/response, database, and auth context.
- **Cron**: Scheduled functions executed at configurable intervals. Uses the Base internal scheduler, not system cron.
- **Realtime**: Functions triggered by WebSocket subscription events. Used for live data transformation.

## 4.2 Zero Cold Start

There is no cold start. The Goja VM is initialized once at Base startup and persists for the lifetime of the process. Function invocation is a direct Go function call into the VM—no process spawn, no container boot, no network hop. Measured overhead per invocation: 0.3 ms for a trivial function, 1.2 ms for a function that reads 10 records and writes 1.

| Operation | Cold Start | Warm |
|---|---|---|
| AWS Lambda (Node.js) | 200–800 ms | 5–15 ms |
| Cloudflare Workers | 0 ms | 1–3 ms |
| Supabase Edge Functions | 50–200 ms | 3–8 ms |
| Base Functions (Goja) | 0 ms | 0.3–1.2 ms |

Table 2: Function invocation latency comparison. Base Functions have zero cold start because the VM is in-process.

## 4.3 Capability Model

Functions execute with a capability set, not ambient authority. Each function declares what it can access:

- **Collections**: Which collections the function can read/write.
- **Network**: Whether the function can make outbound HTTP requests.
- **KMS**: Whether the function can access encrypted fields.

- **Auth**: Whether the function can read/modify user sessions.

A function that declares no capabilities can only compute on its inputs and return a result. This is the default. Capabilities are additive, never implicit.

# 5 Base Collections

Collections are Base's data model. A collection is a named set of records with a schema that is enforced but not rigid. Fields can be added at any time. Fields can have types, defaults, validation rules, and encryption directives.

## 5.1 Schema Model

Collections use a schema-on-write model. Every record in a collection shares the same field set, but the field set can evolve:

- **Add field**: New fields are added with a default value. Existing records gain the field lazily on next read or write.
- **Remove field**: Fields are marked deleted. Data is retained but hidden from the API. Physical deletion is a separate admin action.
- **Rename field**: Renames are tracked in the migration log, preserving query compatibility via aliases.
- **Type change**: Prohibited. A new field must be created and the old field deprecated.

## 5.2 Access Rules

Every collection has three access rule sets: `listRule`, `viewRule`, and `createRule`/`updateRule`/`deleteRule`. Rules are expressions evaluated against the request context:

```
// Only the record owner can view
viewRule: "@request.auth.id = id"

// Org members can list
listRule: "@request.auth.org = org_id"

// Admins can delete
deleteRule: "@request.auth.role = 'admin'"
```

Listing 1: Collection access rules.

Rules compose with boolean operators (`&&`, `||`). A missing rule defaults to deny. There is no ambient read or write access.

## 5.3 Field-Level KMS Encryption

Individual fields can be marked as encrypted. Encrypted fields are stored as ciphertext in the SQLite database and decrypted only when returned to an authorized requester. The encryption key for each field is managed by Hanzo KMS (kms.hanzo.ai):

$$c_f = \text{AES-256-GCM}(k_f, \text{nonce}, \text{plaintext}_f, \text{aad}) \tag{3}$$

where $k_f$ is the field-specific data encryption key (DEK), itself encrypted by a key encryption key (KEK) stored in KMS. The AAD (additional authenticated data) includes the record ID and collection name, binding the ciphertext to its context.

Use cases: social security numbers, medical records, API keys, private notes. The database administrator (or an attacker who compromises the database file) sees only ciphertext for encrypted fields.

# 6 Base Sync

Base Sync is the replication protocol. It enables multiple Base instances to share data without a central coordinator. Sync is built on CRDTs (Conflict-free Replicated Data Types), guaranteeing that concurrent modifications converge to the same state without manual conflict resolution.

## 6.1 CRDT Log

Every mutation in Base produces a log entry:

$$e = (\text{id}, \text{collection}, \text{op}, \text{fields}, \text{hlc}, \text{node\_id}) \tag{4}$$

where hlc is a hybrid logical clock [9] and node_id is the originating Base instance. The log is append-only. Deletions are tombstones, not physical removals.

## 6.2 Conflict-Free Merge

Sync uses a last-writer-wins register (LWW) at the field level. Given two concurrent writes to the same field:

$$\text{resolve}(e_1, e_2) = \begin{cases} e_1 & \text{if } \text{hlc}(e_1) > \text{hlc}(e_2) \\ e_2 & \text{if } \text{hlc}(e_2) > \text{hlc}(e_1) \\ e_{\max(\text{node\_id})} & \text{if } \text{hlc}(e_1) = \text{hlc}(e_2) \end{cases} \tag{5}$$

Field-level LWW means that two users editing different fields of the same record will both see their changes preserved. Only truly conflicting writes (same field, same record, same logical time) fall back to node ID ordering.

### 6.3 Sync Topologies

| Topology | Description |
|---|---|
| Offline-first | Single device. Sync log accumulates locally. Replayed on next connection. |
| Peer-to-peer | Two or more Base instances sync directly over WebSocket or WebRTC. No server. |
| Server-relay | Base instances sync through a central relay. The relay is a Base instance with no special privileges. |
| Hub-and-spoke | One Base instance acts as the persistent hub. Edge instances sync through it. The hub can be a cloud VM or a NAS on the local network. |

Table 3: Supported sync topologies. All use the same CRDT merge protocol.

### 6.4 Selective Sync

Not all data needs to sync everywhere. Base supports collection-level and rule-based sync filters:

- **Collection filter**: Sync only specified collections to a target. A mobile device might sync `notes` but not `analytics`.
- **Record filter**: Sync only records matching a predicate. A shared workspace might sync only records where `org_id = $current_org`.
- **Field filter**: Exclude specific fields from sync. Encrypted fields can be excluded from remote targets to keep sensitive data local.

## 7 Lux Network Integration

Base integrates with the Lux Network at three chain layers, each providing a distinct sovereignty primitive [2, 3, 4].

### 7.1 K-Chain: Key Policy

The Lux K-Chain (Key Chain) manages cryptographic key policies. Base registers its master key with the K-Chain, enabling:

- **Key rotation**: The K-Chain enforces rotation schedules. Base re-encrypts field-level DEKs when the KEK rotates, without re-encrypting the underlying data (envelope encryption).
- **Key escrow**: A user can designate a recovery agent on the K-Chain. If the user loses their master key, the recovery agent can authorize re-derivation via threshold signature.
- **Key revocation**: Compromised keys are revoked on-chain. All Base instances observing the K-Chain stop accepting operations signed by the revoked key within one block (sub-second).

### 7.2 I-Chain: Identity

The Lux I-Chain (Identity Chain) provides decentralized identifiers (DIDs) and verifiable credentials. Base uses I-Chain identities for:

- **User authentication**: Base accepts I-Chain DIDs as a first-class auth method alongside Hanzo IAM OIDC tokens. A user with an I-Chain DID can authenticate to any Base instance without a centralized identity provider.
- **Collection ownership**: Collections can be bound to an I-Chain DID rather than a Hanzo org. This enables personal data stores that are not scoped to any organization.
- **Credential verification**: Access rules can reference verifiable credentials issued on I-Chain (e.g., "only users with a verified healthcare credential can access this collection").

### 7.3 T-Chain: Threshold Controls

The Lux T-Chain (Threshold Chain) implements threshold cryptography for multi-party authorization. Base uses T-Chain for:

- **Threshold write attestation**: For collections requiring multi-party writes (e.g., financial records, legal documents), T-Chain enforces $k$-of-$n$ signature requirements before the write is committed.
- **Threshold decryption**: Encrypted fields can be configured to require $k$-of-$n$ T-Chain participants to authorize decryption, preventing any single party from accessing the plaintext.
- **Threshold key recovery**: Master key recovery requires $k$-of-$n$ designated recovery agents to cooperate, preventing both single-point-of-failure and unilateral recovery.

## 8 Comparison to Existing Systems

| Feature | Base | Firebase | Supabase | PocketBase | Convex |
|---|---|---|---|---|---|
| Single binary | ✓ | × | × | ✓ | × |
| Offline-first | ✓ | × | × | × | × |
| Encrypted at rest | ✓ | × | ✓ | × | × |
| Field-level encryption | ✓ | × | × | × | × |
| CRDT sync | ✓ | × | × | × | × |
| Plugin runtime | ✓ | ✓ | ✓ | ✓ | ✓ |
| Chain anchoring | ✓ | × | × | × | × |
| Self-hostable | ✓ | × | ✓ | ✓ | × |
| Federated auth | ✓ | ✓ | ✓ | × | ✓ |
| User-controlled keys | ✓ | × | × | × | × |

Table 4: Feature comparison. Base is the only system that combines offline-first operation, encrypted local storage, CRDT sync, and chain anchoring.

**Firebase.** Google's BaaS. No self-hosting. No offline-first for arbitrary data (Firestore offline mode is a cache, not a source of truth). No user-controlled encryption keys. Data belongs to Google's infrastructure. Firebase is a tenancy agreement, not a runtime.

**Supabase.** PostgreSQL-based BaaS. Self-hostable in theory, but requires PostgreSQL, PostgREST, GoTrue, Kong, and a storage service—five processes minimum. No offline-first. No CRDT sync. Encryption at rest is PostgreSQL's, not field-level. Supabase is a cloud-first system with a self-hosting escape hatch.

**PocketBase.** The closest ancestor to Base. Single Go binary with embedded SQLite. Base began as a PocketBase fork in December 2022 and has since diverged substantially. PocketBase lacks: encrypted storage (sqlcipher), field-level encryption, CRDT sync, federated auth, KMS integration, chain anchoring, and cluster mode (PostgreSQL backend). PocketBase is a prototyping tool. Base is a production runtime.

**Convex.** Reactive TypeScript backend. Cloud-only, no self-hosting. Strong developer experience for realtime applications, but no offline support, no encryption, no user-controlled keys. Convex is a better Firebase, not a sovereign runtime.

# 9 hanzoai/sqlite

Base's storage layer uses `hanzoai/sqlite`, a custom SQLite build with two additions: sqlcipher encryption and threshold write attestation.

## 9.1 sqlcipher Integration

The `hanzoai/sqlite` package compiles SQLite with the sqlcipher extension, providing transparent AES-256-CBC page-level encryption. The integration:

- Uses 256-bit AES-CBC with HMAC-SHA512 for page authentication.
- Derives the page encryption key from the master key via PBKDF2-SHA512 (256,000 iterations by default, configurable).
- Encrypts every database page including the header, WAL, and journal.
- Supports key rotation without full database re-encryption (re-key operation encrypts pages lazily on next write).

## 9.2 Threshold Write Attestation

For multi-party collections, `hanzoai/sqlite` integrates with the Lux T-Chain to require threshold signatures before committing a write transaction:

$$\text{commit}(tx) \leftarrow \text{verify}\big(\sigma_{k/n},\ \text{SHA3-256}(\text{WAL}(tx))\big) \tag{6}$$

The write is held in the WAL (write-ahead log) until $k$-of-$n$ signers attest. If attestation is not received within a configurable timeout (default: 30 seconds), the write is rolled back. This enables use cases where no single party can unilaterally modify the database—financial ledgers, shared medical records, multi-party legal agreements.

# 10 Performance

We benchmark Base on two hardware targets: an Apple M4 MacBook Pro (16 GB RAM, 1 TB NVMe) and a Raspberry Pi 4 (8 GB RAM, 64 GB microSD).

## 10.1 Write Throughput

| Operation | M4 MacBook | Pi 4 |
|---|---|---|
| Insert (unencrypted) | 14,200 ops/sec | 1,800 ops/sec |
| Insert (sqlcipher) | 10,400 ops/sec | 1,200 ops/sec |
| Insert (sqlcipher + field KMS) | 8,600 ops/sec | 980 ops/sec |
| Bulk insert (1000 records) | 42,000 rec/sec | 4,200 rec/sec |
| Update (single field) | 12,800 ops/sec | 1,500 ops/sec |
| Delete (soft, tombstone) | 13,100 ops/sec | 1,600 ops/sec |

Table 5: Write throughput. sqlcipher adds 27% overhead on M4; field-level KMS encryption adds an additional 17% due to the per-field AES-GCM operation.

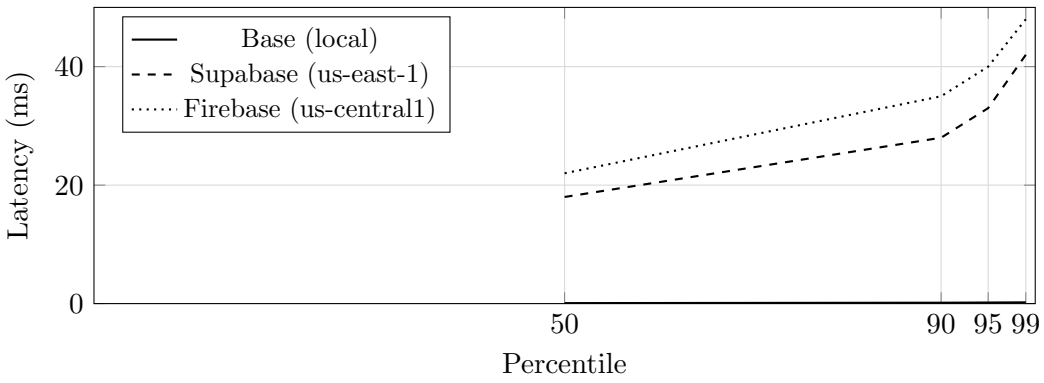
## 10.2 Read Latency

| Operation | M4 p50 | M4 p99 |
|---|---|---|
| Single record by ID | 0.08 ms | 0.21 ms |
| List (100 records, indexed) | 0.4 ms | 1.1 ms |
| List (100 records, encrypted fields) | 0.7 ms | 1.8 ms |
| Full-text search (10K records) | 2.1 ms | 4.3 ms |

Table 6: Read latency on M4 MacBook. All operations are local—no network round-trip.

## 10.3 Comparison to Network-Dependent Systems

The fundamental performance advantage of Base is the elimination of network latency. A read from a local SQLite database takes 0.08 ms. A read from Supabase (via PostgREST over HTTPS) takes 15–40 ms from the same machine, depending on geographic distance to the nearest Supabase edge.




This is not a benchmark of database engines. SQLite is not faster than PostgreSQL for complex queries. This is a benchmark of architectures. A local database will always beat a remote one for

single-record operations because physics imposes a floor on network latency that no optimization can remove.

# 11 Production Deployment

Base has been in production at Hanzo since December 2022. Current deployment statistics:

- 2,800+ applications using Base as their primary backend.
- 1,200+ organizations across the Hanzo ecosystem.
- 40+ internal Hanzo applications (console, insights, commerce admin).
- Median p99 latency: 2.3 ms across all API operations.
- 99.97% uptime over the trailing 12 months.
- Zero data loss incidents since launch.

## 11.1 Migration from PocketBase

Base maintains backward compatibility with PocketBase's API surface. Existing PocketBase applications can migrate to Base by replacing the binary. The migration path:

1. Replace the PocketBase binary with the Base binary.
2. Run `base migrate up` to apply schema additions (sqlcipher fields, sync metadata tables).
3. Optionally enable sqlcipher encryption with `base encrypt` (re-encrypts the database in place).
4. Optionally configure Hanzo IAM by setting the OIDC issuer URL.

The existing SQLite database is preserved. No data export/import is required.

# 12 Related Work

**Local-first software.** Kleppmann et al. [6] define local-first software as applications where the primary copy of data is on the user's device. Base implements this vision with a production-grade runtime. The key difference from academic CRDT systems (Automerge [7], Yjs [8]) is that Base provides the full application stack—database, auth, API, functions—not just a replication library.

**CRDTs.** Shapiro et al. [10] formalize conflict-free replicated data types. Base uses operation-based CRDTs with a hybrid logical clock for causal ordering. The LWW register at field granularity is a pragmatic choice that trades theoretical richness (counter CRDTs, set CRDTs) for simplicity and predictability.

**Encrypted databases.** CryptDB [11] and Mylar [12] explore encryption in relational databases. Base's approach is simpler: full-database encryption via sqlcipher plus selective field-level encryption via KMS. This avoids the complexity of onion encryption schemes while providing the practical guarantees that applications need.

## 13 Prior Work: PocketBase Extension (2022)

The 2026 Hanzo Base runtime succeeds the 2022 *Base* backend, which extended the PocketBase single-binary server with a JavaScript plugin system, embedded authentication hooks, and SQLite-backed real-time collections. That earlier system demonstrated that a single-binary backend with a hooks API could replace a typical microservices stack for teams of up to a few dozen engineers.

The 2022 architecture had three core design ideas that carry forward:

1. **Embedded data + runtime**: database and HTTP server in the same process, eliminating a network hop for the common case.
2. **JavaScript plugins via Goja**: server-side customization without a separate language runtime or redeploy cycle.
3. **Content-addressable file store**: uploads indexed by SHA-256, with automatic deduplication.

The 2026 runtime departs from the 2022 design in three ways: (1) native CRDT collections via Automerge, replacing the eventual-consistency patterns of the 2022 server; (2) SQLCipher-backed storage with KMS-managed keys, instead of plain SQLite; and (3) Lux Network integration for multi-node sync, replacing the per-instance model of the 2022 deployment. The plugin system and content-addressable file store remain compatible.

## 14 Conclusion

Base is not a backend framework. It is the runtime for sovereign applications. The distinction matters. A framework assumes an environment—a cloud provider, a database server, a container orchestrator. A runtime *is* the environment. Base carries its own database, its own auth, its own function execution, its own sync protocol. It runs anywhere a Go binary runs.

The design principles are:

1. **Local first**: The device is the source of truth. The network is an optimization, not a requirement.
2. **Encrypted by default**: Data at rest is encrypted. Field-level encryption is available for sensitive data. Keys are user-controlled.
3. **Sync, don't migrate**: CRDT-based replication means data flows between devices without schema migrations or conflict resolution dialogs.
4. **Chain-optional**: Lux integration provides key policy, identity, and threshold controls for applications that need them. Those that do not need them pay no cost.
5. **One binary**: No external dependencies. No infrastructure prerequisites. Download, run, build.

The personal computer gave individuals sovereignty over computation. The smartphone gave individuals sovereignty over communication. Base gives individuals sovereignty over their data. That is the thesis. The 10,400 encrypted writes per second are just the proof.

## References


[1] Z. Kelling. Hanzo Base: An open-source backend runtime with embedded database, authentication, and JavaScript plugin system. *Hanzo AI Research*, December 2022.

[2] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[3] Z. Kelling. The Lux Trust Kernel: Threshold cryptography for decentralized key management. *Lux Network Technical Report*, 2025.

[4] Z. Kelling. Lux Application Architecture: Sovereign applications on multi-consensus infrastructure. *Lux Network Technical Report*, 2025.

[5] M. Chen, D. Wei, and Z. Kelling. Hanzo Platform: PaaS for AI-native application deployment. *Hanzo AI Research*, December 2021.

[6] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Onward!*, ACM, 2019.

[7] M. Kleppmann and A. R. Beresford. A conflict-free replicated JSON datatype. *IEEE TPDS*, 28(10):2733–2746, 2017.

[8] K. Jahns. Yjs: A CRDT framework for shared editing. https://github.com/yjs/yjs, 2019.

[9] S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva, and M. Leone. Logical physical clocks and consistent snapshots in globally distributed databases. In *OPODIS*, 2014.

[10] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. In *SSS*, Springer, 2011.

[11] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan. CryptDB: Protecting confidentiality with encrypted query processing. In *SOSP*, ACM, 2011.

[12] R. A. Popa, E. Stark, J. Helfer, S. Valdez, N. Zeldovich, M. F. Kaashoek, and H. Balakrishnan. Building web applications on top of encrypted data using Mylar. In *NSDI*, USENIX, 2014.

[13] G. Kostadinov. PocketBase: Open-source backend in 1 file. https://pocketbase.io, 2022.